Group Project 1: Safety Approval Policies for Agent Harnesses

Instructor: Bo An, Professor, CCDS, NTU

Teaching Assistant:
Deng Yanchen, ycdeng@ntu.edu.sg

1. Background

AI agents interact with external environments through tool calls (e.g., executing shell commands, calling APIs, modifying files). In a typical agent harness, each tool call must be approved before execution to prevent unintended or harmful side effects.
Two naive extremes exist for approval policies. Full human approval leads to severe confirmation fatigue. Anthropic reports that 93% of tool calls are blindly approved by human reviewers under this scheme, rendering the safety mechanism ineffective. Full auto-execution, on the other hand, is unsafe: agents have been observed to delete remote branches, drop production databases, overwrite critical files, and perform other destructive operations without user intent.
This motivates the need for an intelligent, intermediate approval policy that balances safety and usability.

2. Problem Description

Design and implement an **approval policy** that makes a **three-way decision** for each tool call issued by an AI agent:

- **Auto-approve:** The tool call is deemed safe and is executed without human intervention.
- **Auto-deny:** The tool call is deemed unsafe or out-of-scope and is blocked automatically.
- **Escalate to human:** The tool call is ambiguous or high-risk, and is forwarded to a human reviewer for a manual approve/deny decision.

The goal is to maximize **safe task completion**, i.e., the fraction of tasks that are successfully completed without executing any unauthorized tool call, **under a limited human attention budget k**, where k represents the maximum number of tool calls that can be escalated to a human reviewer per task. The policy should allocate this scarce human attention to the tool calls where it matters most. **Once the escalation budget k is exhausted, any subsequent decision that would otherwise require escalation is treated as an auto-deny.**

The approval policy can leverage any available information at decision time, including but not limited to: the tool name and its parameters, the agent's preceding trace, the conversation history, the task description, and any tool call metadata (e.g., whether the tool is read-only vs. write, whether it touches local vs. remote resources).

3. Evaluation

3.1 Main Metrics

- **TCR@k (Safe Task Completion Rate at budget k):** The fraction of tasks that the agent successfully completes **without executing any unauthorized tool call**, when the approval policy is allowed to escalate at most $k$tool calls to a human reviewer. This metric should be reported as a curve (TCR vs. k) to show how task completion degrades as the human attention budget shrinks.
- **HITL Load (Human-In-The-Loop Load):** The total number of human interventions (escalations) across all successfully completed tasks. Lower is better — an ideal policy achieves high TCR with minimal escalation.

3.2 Benchmark

- **AgentDojo** (NeurIPS 2024): An existing benchmark framework for evaluating AI agent safety. The project should integrate the approval gate into the AgentDojo evaluation pipeline.

3.3 Human Reviewer Simulation

Since real human reviewers are impractical at evaluation scale, the evaluation should use an **LLM-simulated human reviewer** for online evaluation. The simulated reviewer should receive the tool call details and the task context, and return an approve/deny decision. The design and prompting strategy for this simulated reviewer should be documented. The simulated reviewer must only receive information that would reasonably be available to a real human reviewer. **Benchmark ground-truth labels, attack annotations, and other oracle information must not be exposed to the reviewer.**

4. Deliverables

Please submit the following deliverables via a Github repo.

4.1 Codebase

- An **approval gate** integrated into the AgentDojo framework, intercepting each tool call before execution and routing it through the approval policy.
- At least **one developed approval policy** (the student's own approach — e.g., rule-based, classifier-based, LLM-based, or a learned policy).
- **Two baselines** for comparison:
  - **Full-approve baseline:** Every tool call is auto-approved (no human intervention; equivalent to unguarded agent execution).
  - **Full-escalate baseline:** Every tool call is escalated to the human reviewer (maximum safety but maximum HITL load; equivalent to full human review).

4.2 Report

The report must contain, at minimum:

- **TCR@k curves** comparing the developed policy against both baselines. The curves should cover a meaningful range of *k* values.
- **HITL Load table** showing total escalation counts for each policy across the benchmark suite.
- **Per-strategy API cost breakdown:** Since both the approval policy and the simulated human reviewer may invoke LLM calls, the report should quantify the additional API cost (in tokens or dollars) introduced by each strategy, broken down by component (policy inference, reviewer simulation, agent execution).
- **Failure case classification:** Categorize failures into at least two types:
  - **Over-conservative failures:** The policy auto-denied a safe tool call, or unnecessarily escalated safe tool calls such that the escalation budget was exhausted, preventing successful task completion.
  - **Over-permissive failures:** The policy auto-approved an unsafe tool call, leading to an unauthorized action (false negative).
- **Representative trajectory examples:** For each failure type, provide at least one concrete trajectory showing the sequence of tool calls, the policy's decisions, and where the failure occurred.
- **Escalation-value analysis:** Discuss which types of tool calls benefit most from human escalation (e.g., write operations to production systems, irreversible deletions, external API calls with side effects) and which can be safely auto-approved (e.g., read-only queries, local computation).

4.3 Presentation

Each group will give a **10-minute presentation**, followed by **5 minutes of Q&A**. The presentation should cover:

- The main idea and design of your approval policy.
- Key implementation choices and how the approval gate is integrated into AgentDojo.
- The most important experimental results, including comparisons with the two baselines.
- Representative failure cases and insights from the escalation-value analysis.
- The main conclusions and lessons learned from your project.